Match score model:
Poisson goals, outcomes, and empirical rates

Abstract

We specify independent Poisson goals for two teams, record the joint distribution and match-outcome probabilities, then connect the Poisson means (λ_A, λ_B) to abstract attack/defense strengths and to simple plug-in formulas using per-team goals for/against and a league baseline.

1 Match-level model: Poisson goals

For one fixture, let X_A and X_B denote goals scored by teams A and B over a fixed period (e.g. regulation time). We treat these as random counts.

1.1 Rates

Each team can be described by an *attack* strength and a *defense* strength (from data, priors, or ratings). The *expected* goals in the match are driven by how each side's attack meets the opponent's defense:

- λ_A reflects **A's attack against B's defense** (how well A scores when facing B's defensive quality).
- λ_B reflects **B's attack against A's defense**.

Abstractly one chooses a map from those four ingredients into two nonnegative numbers, e.g.

$$\lambda_A = f(\mathrm{atk}_A, \mathrm{def}_B), \qquad \lambda_B = f(\mathrm{atk}_B, \mathrm{def}_A),$$

with the exact form of f (linear, log-linear, tabulated, etc.) left to the modeller.

1.2 Independence and joint distribution

Assume

$$X_A \sim \mathrm{Poisson}(\lambda_A), \qquad X_B \sim \mathrm{Poisson}(\lambda_B), \qquad X_A \perp X_B \ \text{ given } (\lambda_A, \lambda_B). \tag{1}$$

Then the **joint probability mass function** is, for nonnegative integers i, j,

$$\mathbb{P}(X_A = i,\, X_B = j) = e^{-(\lambda_A+\lambda_B)}\, \frac{\lambda_A^i}{i!}\, \frac{\lambda_B^j}{j!}. \tag{2}$$

Computation. For implementation, one often truncates to a finite goal grid (e.g. $i, j \in \{0, \ldots, K\}$) and renormalizes the restricted distribution if probabilities outside the grid are negligible.

1.3 Match outcomes

From (2),

$$\begin{aligned}\mathbb{P}(A \text{ wins}) &= \mathbb{P}(X_A > X_B),\\ \mathbb{P}(\text{draw}) &= \mathbb{P}(X_A = X_B),\\ \mathbb{P}(B \text{ wins}) &= \mathbb{P}(X_B > X_A).\end{aligned}$$

These three probabilities sum to 1.

Summary of Section 1. Pick (λ_A, λ_B) from **A's attack vs B's defense** and **B's attack vs A's defense**, then use independent Poisson goals to obtain the full score distribution and win/draw/lose probabilities.

2 Interpreting λ as expectations

For a fixture between A and B, one may interpret

$$\lambda_A = \mathbb{E}[X_A \mid \text{match } (A, B)], \qquad \lambda_B = \mathbb{E}[X_B \mid \text{match } (A, B)],$$

when the Poisson model is taken literally. The rest of the note discusses *constructing* numerical values for λ_A, λ_B from simple league summaries.

3 Notation for empirical summaries

From past results (same competition and time window):

- $\bar{s}_i$: mean goals *for* team i per match (GF).
- $\bar{c}_i$: mean goals *against* team i per match (GA).
- μ: league mean goals scored per team–match row. With a balanced schedule and full double counting of each fixture, this typically equals the league mean GA per row as well.

4 Multiplicative attack $\times$ opponent defense

A simple closed form that adapts λ to the opponent is

$$\lambda_A \;=\; \bar{s}_A \cdot \frac{\bar{c}_B}{\mu}, \qquad \lambda_B \;=\; \bar{s}_B \cdot \frac{\bar{c}_A}{\mu}. \tag{3}$$

Interpretation. $\bar{s}_A$ is how much A scores against an "average" defense in the sample. The factor $\bar{c}_B/\mu$ measures how much *more* (or less) than average B concedes. If B is average defensively, $\bar{c}_B = \mu$ and $\lambda_A = \bar{s}_A$.

Equivalent form.

$$\lambda_A \;=\; \mu \cdot \frac{\bar{s}_A}{\mu} \cdot \frac{\bar{c}_B}{\mu},$$

i.e. baseline μ times an attack index for A times a defensive leakiness index for B.

Link to Section 1. This choice of f uses *observed* GF/GA as proxies for attack and opponent-relative defense; it is one convenient plug-in, not the only map from $(\text{atk}_A, \text{def}_B)$ to λ_A.

5 Log-linear (Dixon–Coles style) view

A standard parameterization is

$$\log \lambda_{ij} \;=\; \log \mu + \alpha_i + \delta_j,$$

where α_i is an attack effect and δ_j a defense effect on goals opponents score against j. Plug-in estimates from summaries (before centering) give $\alpha_i \approx \log(\bar{s}_i/\mu)$, $\delta_j \approx \log(\bar{c}_j/\mu)$, hence

$$\lambda_A \approx \mu \, \exp(\alpha_A + \delta_B),$$

consistent with (3) when exponentials are expanded without centering constraints. In a full fitted model, $\{\alpha_i\}$ and $\{\delta_j\}$ are usually centered for identifiability.

6 Shrinkage

Raw $\bar{s}_i$ and $\bar{c}_i$ are noisy when the sample size n_i is small. An empirical-Bayes style fix shrinks toward the league mean, e.g.

$$\tilde{s}_i \;=\; \frac{n_i}{n_i + k}\,\bar{s}_i + \frac{k}{n_i + k}\,\mu$$

(and similarly $\tilde{c}_i$), then substitute $\tilde{s}$ and $\tilde{c}$ into (3). The constant $k > 0$ controls shrinkage strength.

7 Further remarks

- **Consistency with** (1)**.** Under independent Poissons, λ_A and λ_B are not forced to sum to a fixed number; richer dependence (e.g. Dixon–Coles adjustment for low scores) modifies the joint law beyond (2).
- **Home advantage.** If desired, multiply the relevant λ by a factor $h > 1$ for the home side (or add $\log h$ in log space).
- **Schedule strength.** Raw $\bar{s}_i, \bar{c}_i$ mix team ability with the quality of opponents already played; iterative or likelihood-based attack–defense models partially address this.
- **Relation to** (2)**.** Any (λ_A, λ_B)—whether from theory, ratings, or (3)—can be inserted into (2) to approximate the score grid and outcome probabilities in Section 1.

8 From match-level Poisson rates to league simulation

This repo uses the same Poisson goals model to simulate an entire league season in a week-by-week (autoregressive) way.

8.1 When a scoreline is missing

The input table stores, for each fixture cell, an encoded opponent index, a venue flag (home/away), and the realized scoreline from the perspective of the row team, with an explicit placeholder ? for not-yet-played games. The notebook detects the first week containing any ? cell and treats all earlier weeks as already played.

8.2 Autoregressive Monte Carlo

For each Monte Carlo run, the simulation iterates weeks from that first missing week to the last week column. Before simulating a given week, it maintains pooled, per-team scoring summaries computed from all games known so far in that run:

- pooled goals-for for team t: $\bar{s}_t$,
- pooled goals-against for team t: $\bar{c}_t$,

and it computes a league baseline μ from the same set of known matches (overall mean goals scored per match-row).

For an upcoming fixture, the notebook knows the host/guest from the venue flag in the input and converts pooled summaries into Poisson mean goal rates using the multiplicative structure of Eq. (3), with a fixed home multiplier $h > 1$ for the host side:

$$\lambda_{\text{host}} = \bar{s}_{\text{host}} \cdot \frac{\bar{c}_{\text{guest}}}{\mu} \cdot h,$$
$$\lambda_{\text{guest}} = \bar{s}_{\text{guest}} \cdot \frac{\bar{c}_{\text{host}}}{\mu}.$$

Given these rates, it draws independent Poisson goals $X_{\text{host}} \sim \text{Poisson}(\lambda_{\text{host}})$ and $X_{\text{guest}} \sim \text{Poisson}(\lambda_{\text{guest}})$. If the fixture score is already present (not ?), the realized scoreline is used instead of sampling. After completing each full week, it updates the pooled summaries with the newly known (realized or simulated) outcomes, so later weeks are conditional on earlier simulated results.

8.3 League table, tie-breaks, and rank probabilities

After simulating through the last week, the notebook computes final league points (3 for a win, 1 for a draw) and orders teams using: points, then head-to-head points among tied teams, then head-to-head goal difference among those teams, then head-to-head goals scored. If metrics still do not uniquely determine an ordering, it falls back to a deterministic order by team index to prevent Monte Carlo crashes.

Finally, it estimates $P(\text{rank} = k)$ by counting how often each team finishes at each rank across all Monte Carlo runs and reports an expected rank as the probability-weighted average.